Reviewer Pack — Minimal Hodge Index in Configuration Space and Communication…

Entry 1202733ea286 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 10:17:08 UTC

Minimal Hodge Index in Configuration Space and Communication Complexity Rank Variance

Entry ID: `1202733ea286` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 10:17:08 UTC

1. Conjecture

Field A (mathematical branch): Configuration Space Geometry **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every k-clique with n vertices, the minimal Hodge index of its associated configuration space is linearly correlated with the communication complexity rank variance R^k(V), where V is the vector space corresponding to the k-clique. Specifically, the minimal Hodge index h_min(C) is Θ(R^k(V)).

Rationale (proposer's reasoning):

> Configuration spaces are geometric objects that encode various combinatorial properties of graphs. The Hodge index captures certain topological features of these spaces and has been previously applied in geometry and physics. Communication complexity rank variance measures the difficulty of distributing information among parties, a key concept in distributed computing. This conjecture suggests a potential link between geometric and computational complexities, which could provide insights into both fields.

Taxonomy category: `ConfigurationSpaceGeometry × CommunicationComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2e9c6dcd4a0184bc`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between minimal Hodge index and communication complexity rank variance for all k-cliques with n vertices (n ≤ 40) across 30 random seeds exceeds 0.8, with no seed producing a correlation coefficient less than 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal Hodge Index configuration space AND communication complexity rank variance` - `configuration space geometry application in communication complexity rank variance` - `linear correlation between Hodge index and R^k(V) vector space`

Top relevant hits considered: - [http://arxiv.org/abs/1803.01936v4] Variations of Hodge Structures of Rank Three k-Higgs Bundles and Moduli Spaces of Holomorphic Triples - [http://arxiv.org/abs/1906.03766v3] Variance Reduction in Gradient Exploration for Online Learning to Rank - [http://arxiv.org/abs/1811.03413v1] Optical communication on CubeSats - Enabling the next era in space science - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics - [http://arxiv.org/abs/2202.08970v1] Status and initial physics performance studies of the MPD experiment at NICA - [http://arxiv.org/abs/1511.08039v2] Measurement of forward W and Z boson production in pp collisions at $\sqrt{s} = 8\,\text{TeV}$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique(k, n):
        if k > n or n < 2:
            return None
        vertices = list(range(n))
        clique = []
        for _ in range(k):
            v = random.choice(vertices)
            clique.append(v)
            vertices.remove(v)
        return clique

    def matrix_multiplication(A, B):
```

```
    m, p = len(A), len(B[0])
    n = len(B)
    result = [[sum(A[i][k] * B[k][j] for k in range(n)) for j in range(p)] for i in range(m)]
    return result

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        max_row = i
        for j in range(i + 1, rows):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        factor = matrix[i][i]
        for j in range(cols):
            matrix[i][j] /= factor
        for j in range(rows):
            if i != j:
                factor = matrix[j][i]
                for k in range(cols):
                    matrix[j][k] -= factor * matrix[i][k]
    return matrix

def rank(matrix):
    rref = gaussian_elimination(matrix)
    rank = 0
    for row in rref:
        if any(row):
            rank += 1
    return rank

def hodge_index(n):
    # Example Hodge index calculation (simplified)
    return n * (n - 1) // 2

instances_tested = 0
total_hodge_index = 0
total_variance = 0
n_max = 5

for n in [5, 10, 15, 20, 30, 40]:
    if n > n_max:
        n_max = n

    for _ in range(5):
        k = random.randint(2, min(n - 1, 6))
        clique = generate_k_clique(k, n)
        if not clique:
            continue

        instances_tested += 1
        h_index = hodge_index(n)
        total_hodge_index += h_index

        V = [[0] * k for _ in range(k)]
        for i in range(k):
```

```
                for j in range(i + 1, k):
                    V[i][j] = V[j][i] = random.randint(1, n)

            variance = rank(V)
            total_variance += variance

    if instances_tested == 0:
        return {
            "metric_name": "Hodge Index vs Variance",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "No valid instances generated"
        }

    mean_hodge_index = Fraction(total_hodge_index, instances_tested)
    mean_variance = total_variance / instances_tested

    correlation_coefficient = (instances_tested * sum(h * v for h, v in zip(mean_hodge_index.numer
                               mean_hodge_index.numerator * mean_variance.numerator) / \
                              math.sqrt((instances_tested * sum(h**2 for h in mean_hodge_index.num
                                        (instances_tested * sum(v**2 for v in mean_variance.numera

    return {
        "metric_name": "Hodge Index vs Variance",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": correlation_coefficient > 0.8 and all(correlation_coefficient >= 0.5 f
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_value = sum(r['metric_value'] for r in results if r['metric_value'] is not None) / len(re
    std_value = math.sqrt(sum((r['metric_value'] - mean_value)**2 for r in results if r['metric_va
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r['conjecture_holds'] for r in results) and min(r['metric_value'] for r in result
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={seeds[next(i for i, r in
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ca0f174.py", line 128, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ca0f174.py", line 108, in run_trial
    correlation_coefficient = (instances_tested * sum(h * v for h, v in zip(mean_hodge_index.numerator,
                                                                            ^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'float' object has no attribute 'numerator'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support conditions. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17044
2	propose	ollama_remote	glm4:latest	0	0	13161
3	preregistration	ollama_remote	glm4:latest	0	0	9242
4	novelty	ollama_remote	glm4:latest	0	0	8505
5	novelty	ollama_remote	glm4:latest	0	0	10067
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21315
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12388
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11390
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16444
10	judge	ollama_remote	glm4:latest	0	0	17035

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 136590 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/1202733ea286.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1202733ea286.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1202733ea286.tar.gz` (if generated)